We Build Trades RAG Assistant

Stage-by-stage learning guide - simple chat prototype

Run commands from H:\WBT_RAG in PowerShell. Keep .env private. This guide records implementation progress, not approval of WBT business claims.

Stage 1 - Requirements and source register

Goal: Define what the business assistant should answer and which documents it may use.

New concept: A source register is a list of approved input locations and their labels. It is not itself evidence for a business claim.

Files: PROJECT_BRIEF.md, AGENTS.md, data/sources.json

What we built: The brief requires cited answers, distinct UK/US terms, and clear uncertainty. The register records source IDs, region, origin, status, and cautions. The PDFs are WBT-provided; current validity remains separate.

How to check: Open data/sources.json. Confirm each PDF has a path and status, and website links are restricted to the intended first-MVP set.

```
notepad .\data\sources.json
```

Expected result: You can identify the registered PDFs and their source-status labels.

Stage 2 - Hosted model connection

Goal: Prove that the app can send a simple question to Groq and receive an answer.

New concept: An LLM generates text from a prompt. The connection check is not RAG yet because it does not retrieve WBT documents.

Files: main.py, src/config.py, .env.example, requirements.txt

What we built: The app uses LangChain's ChatGroq. The key lives in an uncommitted .env file. The model receives only the sample CRM question in this check.

How to check: Run the command below after configuring GROQ_API_KEY. Read the answer, request time, and token counts. This command makes a live hosted-model call.

```
.\.venv\Scripts\python.exe main.py
```

Expected result: A short CRM explanation and timing/token information. A prior run took 0.53 seconds; speed can vary.

Stage 3 - Extract PDF pages

Goal: Turn registered PDFs into readable text while retaining source and page information.

New concept: Extraction copies machine-readable text from a PDF. It does not verify whether a business statement is current or approved.

Files: src/extract_pdf.py, data/sources.json, data/processed/*_pages.json, data/processed/*_preview.txt

What we built: Each page becomes a LangChain Document with page number, WBT-provided origin, and source metadata. Five PDFs are registered, including the £47 heating and £297 TBS offer scripts. The script excludes the specified personal seed-PDF passage and records a file fingerprint.

How to check: Open the preview and compare sample prices, currencies, and headings against the PDF. Keep extraction_review pending until a human has actually reviewed it.

```
.\.venv\Scripts\python.exe -m src.extract_pdf --source-id wbt-content-strategy
```

Expected result: Six content-strategy pages extracted locally. The two offer scripts add five and four pages. Current validity remains unconfirmed; no model call is made.

Still to verify: The strategy PDF and seed contain conflicting price/currency and story details. Neither is automatically current policy.

Stage 4 - Split pages into searchable passages

Goal: Prepare short passages that a later search step can find and cite.

New concept: A chunk is a text passage. A 200-character overlap repeats some text at boundaries so a sentence is less likely to lose context. These settings count characters, not model tokens.

Files: src/chunk_documents.py, data/processed/chunks.json, data/processed/parent_pages.json, data/processed/chunks_preview.txt

What we built: RecursiveCharacterTextSplitter uses 900-character chunks and 200-character overlap. The original 1,000-character size exceeded MiniLM's token limit for two newly added passages. It splits each PDF page separately and records exact offsets. Complete parent pages are saved for context and citations.

How to check: Run the command and inspect chunks_preview.txt. Check one passage's page number, region, status, and source caution. Then run the offline tests.

```
.\.venv\Scripts\python.exe -m src.chunk_documents

.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Expected result: The script reports chunk and page counts, writes three files, and makes no model call. Tests finish with OK.

Still to verify: Visual extraction review is still pending. Stage 5 checks the embedding model's token limit before indexing these chunks.

Stage 5 - Local embedding and search

Goal: Find relevant PDF passages without sending WBT text to a hosted model.

New concept: An embedding is a numeric representation of text. Vector search finds related meaning; keyword search helps with exact terms. Search results are evidence candidates, not answers.

Files: src/index.py, src/retrieval.py, data/index/chroma, data/index/manifest.json, requirements.txt

What we built: MiniLM runs on CPU. Before indexing, its tokenizer checks every full chunk against the model's input limit so no passage is silently truncated. Chroma persists vectors locally; retrieval combines vector and keyword rankings.

How to check: Install dependencies, build the index, and search for Trade Business School. Then try --region UK to see exact region filtering. The first model load downloads from Hugging Face.

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m src.index

.\.venv\Scripts\python.exe -m src.retrieval "What is Trade Business School?"
```

Expected result: The index reports token counts and passage count. Search shows PDF title, page, status, caution, and passage text. No Groq call is made.

Still to verify: WBT provided these PDFs, but current validity and text extraction review are still pending. UK/US web pages have not been indexed; an exact region filter can correctly return no evidence.

Stage 6 - Grounded answer generation

Goal: Answer a staff question using retrieved PDF evidence and readable citations.

New concept: Grounding means the model receives selected source passages with the question. A citation points to evidence; it does not by itself prove a claim is current or correct.

Files: src/answer.py, src/retrieval.py, data/processed/parent_pages.json, tests/test_answer.py, README.md

What we built: Search finds passages; the answer stage checks their full parent PDF pages and sends those pages to Groq. The prompt requires citations and ignores instructions inside documents. Missing/invalid citation numbers are rejected. Source IDs stay internal.

How to check: Run the TBS question and verify the answer has [1] beside factual claims plus a PDF title and page. Try the UK-price command; it should stop before Groq because no UK-labelled PDF is indexed. Offline tests use fake responses.

```
.\.venv\Scripts\python.exe -m src.answer "What is Trade Business School?"

.\.venv\Scripts\python.exe -m src.answer "What are the UK blueprint prices?" --region UK

.\.venv\Scripts\python.exe -m unittest discover -s tests -q
```

Expected result: The TBS command makes one Groq call and displays a short cited answer. The UK command says the indexed sources do not establish an answer and makes no Groq call. Tests finish with OK.

Still to verify: The TBS call sends selected WBT PDF text to Groq. WBT-provided origin does not establish current validity. Model citations still need evaluation against the original PDFs.

Stage 7 - Local Streamlit interface

Goal: Ask questions in a browser and inspect the evidence behind each cited answer.

New concept: A user interface collects the question and region, then displays the answer and source pages. A form submits only when Ask is pressed; the local embedding model is reused while the app stays open.

Files: app.py, src/answer.py, src/retrieval.py, tests/test_app.py, requirements.txt, README.md

What we built: The page provides a question box, UK/US/North America selection, cited answers, and expandable PDF pages. A missing region or missing evidence is shown without a Groq call. Source IDs stay internal.

How to check: Start Streamlit on localhost. Ask what Trade Business School is and expand a cited page. Select UK and ask about blueprint prices. Stop with Ctrl+C.

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m streamlit run app.py --server.address 127.0.0.1
```

Expected result: At http://localhost:8501, the TBS answer has citations and expandable PDF evidence. The UK pricing question reports insufficient indexed evidence without a Groq call.

Still to verify: This is a local prototype without staff login. The first live TBS question sends selected PDF text to Groq. Answer and citation accuracy still need human review; website pages have not been indexed.

Stage 8 - Offline evaluation and offer-script coverage

Goal: Check retrieval and answer guards against repeatable WBT questions after adding two more PDFs.

New concept: An evaluation set pairs questions with expected results. Passing retrieval checks means the expected evidence was found; it does not prove a generated answer is correct.

Files: data/sources.json, data/source_docs/, src/chunk_documents.py, src/evaluate.py, eval/questions.json, eval/report.json, eval/REVIEW.md

What we built: The £47 heating and £297 TBS sales-script PDFs are registered with unspecified region and unconfirmed current terms. Chunk size is 900 characters with 200 overlap after two 1,000-character chunks exceeded MiniLM's limit. The index has 102 chunks from 36 pages across five PDFs. Eleven offline checks cover new offers, refunds, conflicting currencies, UK/US isolation, and current-price refusal.

How to check: Run the offline evaluation and unit tests. Read the report and compare the new extraction previews with the original PDFs. The evaluation makes no Groq call.

```
.\.venv\Scripts\python.exe -m src.evaluate

.\.venv\Scripts\python.exe -m unittest discover -s tests -q

notepad .\eval\REVIEW.md
```

Expected result: The evaluation reports 11/11 checks passed and writes eval/report.json; unit tests report OK. Current offer policy, visual extraction, and live answer accuracy remain for review.

Still to verify: The user later authorized sending WBT source data to Groq for prototype validation. The PDFs still need visual and business review.

Stage 9 - Official website sources and first complete prototype

Goal: Add the seven registered public WBT pages and validate PDF and URL citations in the local app.

New concept: An allowlist limits fetching to named URLs. A retrieval timestamp records when a page was observed; it does not prove a price or policy has been approved as current.

Files: src/extract_web.py, src/chunk_documents.py, src/index.py, src/retrieval.py, src/answer.py, app.py, .streamlit/config.toml, data/processed/web_fetch_report.json, eval/questions.json, README.md

What we built: The fetcher reads robots.txt, visits only the seven registered pages, omits forms and scripts, and records failures. All seven pages returned HTTP 200 on the successful live run. Their titles, URLs, regions, timestamps, and cautions travel with each passage. The local index now has 132 chunks across five PDFs and seven pages. UK and US pages remain separate. Groq receives only selected evidence after a question is submitted. Streamlit usage telemetry is disabled to avoid a Windows profile write error.

How to check: Fetch, split, index, run offline checks, then launch the app. Ask what TBS is, then ask about a US Electrical Funnel Blueprint with US selected. Expand each citation. The live answers send selected source text to Groq.

```
.\.venv\Scripts\python.exe -m src.extract_web

.\.venv\Scripts\python.exe -m src.chunk_documents

.\.venv\Scripts\python.exe -m src.index

.\.venv\Scripts\python.exe -m src.evaluate

.\.venv\Scripts\python.exe -m streamlit run app.py --server.address 127.0.0.1
```

Expected result: Seven fetched pages; 132 chunks if the site stays unchanged; 14/14 offline checks pass. The app opens at localhost:8501 and shows PDF page or website URL evidence. Site content and counts can change on later fetches.

Still to verify: The first live TBS and US-page answers returned citations, but human claim-by-claim review and current-policy approval remain open. Current price requests are withheld until a source is marked current-approved.

Stage 10 - Branded knowledge desk and clear application errors

Goal: Make the local interface feel like a WBT business tool and show expected problems in plain language.

New concept: A project exception names an expected setup or source-validation failure. It does not remove the safety check; the interface can show the message without a traceback.

Files: app.py, .streamlit/config.toml, src/errors.py, src/answer.py, src/retrieval.py, src/index.py, src/extract_pdf.py, src/extract_web.py, src/chunk_documents.py, tests/test_app.py, README.md

What we built: The interface uses the current public site's deep navy, teal, aqua, and Inter styling. It has a search area, region selector, evidence-led answer, and expandable source trail. Expected project failures now use WBTError rather than a generic built-in exception. Source checks and answer guards still run. The search index and documents did not change.

How to check: Restart Streamlit from your own PowerShell terminal and open 127.0.0.1:8501. Ask how WBT and TBS work together and inspect the source trail. Run the offline test command to check behavior without a Groq call.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q

.\.venv\Scripts\python.exe -m src.evaluate

.\.venv\Scripts\python.exe -m streamlit run app.py --server.address 127.0.0.1
```

Expected result: 31 unit tests pass, 14/14 offline checks pass, and the WBT-styled page loads with the existing cited-answer workflow. A submitted live question still sends selected source text to Groq.

Still to verify: The visual design has not yet had user review. Citation accuracy and current business policy still need human review before staff deployment.

Stage 11 - Simple chat interface

Goal: Make the interface calm and minimal while keeping answers traceable.

New concept: A chat display can retain messages locally without sending prior turns to the model. For now, each submitted question is independently retrieved and answered.

Files: app.py, .streamlit/config.toml, src/answer.py, tests/test_app.py, tests/test_answer.py, README.md

What we built: The interface has a single chat input that sends on Enter and one New chat control. The large source section, search button, and region selector are gone. PDF title/page and official-page URL citations appear inline in answers. Region words in questions guide retrieval; ambiguous offer questions ask for UK or US. New chat clears only local displayed messages.

How to check: Restart the server, type a question, and press Enter. Ask a second question, then select New chat. Confirm the conversation disappears and a new question can be entered. Run offline tests without a Groq call.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q

.\.venv\Scripts\python.exe -m src.evaluate

.\.venv\Scripts\python.exe -m streamlit run app.py --server.address 127.0.0.1
```

Expected result: 33 unit tests pass and 14/14 offline checks pass. The page shows one message field and New chat; cited answers show inline document pages or website links.

Still to verify: The UI shows earlier messages, but follow-up questions do not yet use them as context. The next stage can add safe conversational follow-ups if needed.